

Lakeflow Spark Declarative Pipeline (LDP)

Data Engineering in Databricks



databricks LAKEFLOW

UNIFIED DATA ENGINEERING FOR THE DATA INTELLIGENCE PLATFORM

CONNECT



Efficient
ingestion
connectors

SPARK DECLARATIVE PIPELINES (Previously DLT)



Accelerated ETL
development

JOBS

(Previously Workflows)



Reliable
orchestration for
analytics and AI

INDUSTRY LEADING DATA PROCESSING ENGINE

(Apache Spark + Structured Streaming)

UNIFIED GOVERNANCE



Unity Catalog

OPTIMIZED STORAGE



DELTA LAKE



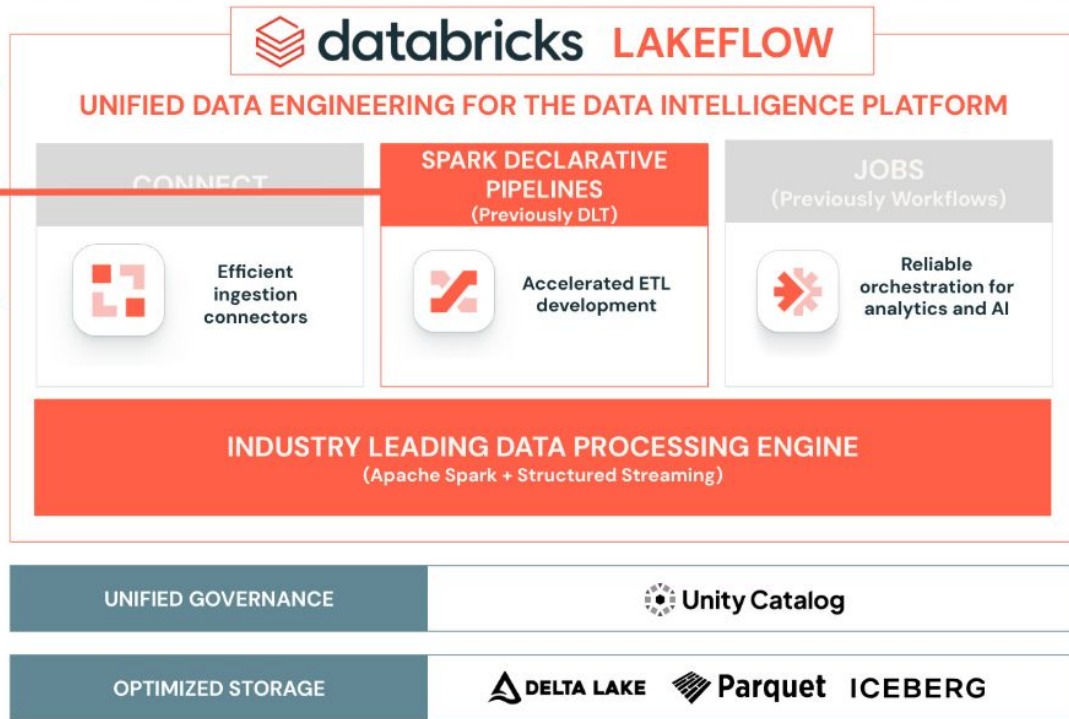
Parquet ICEBERG



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Data Engineering in Databricks

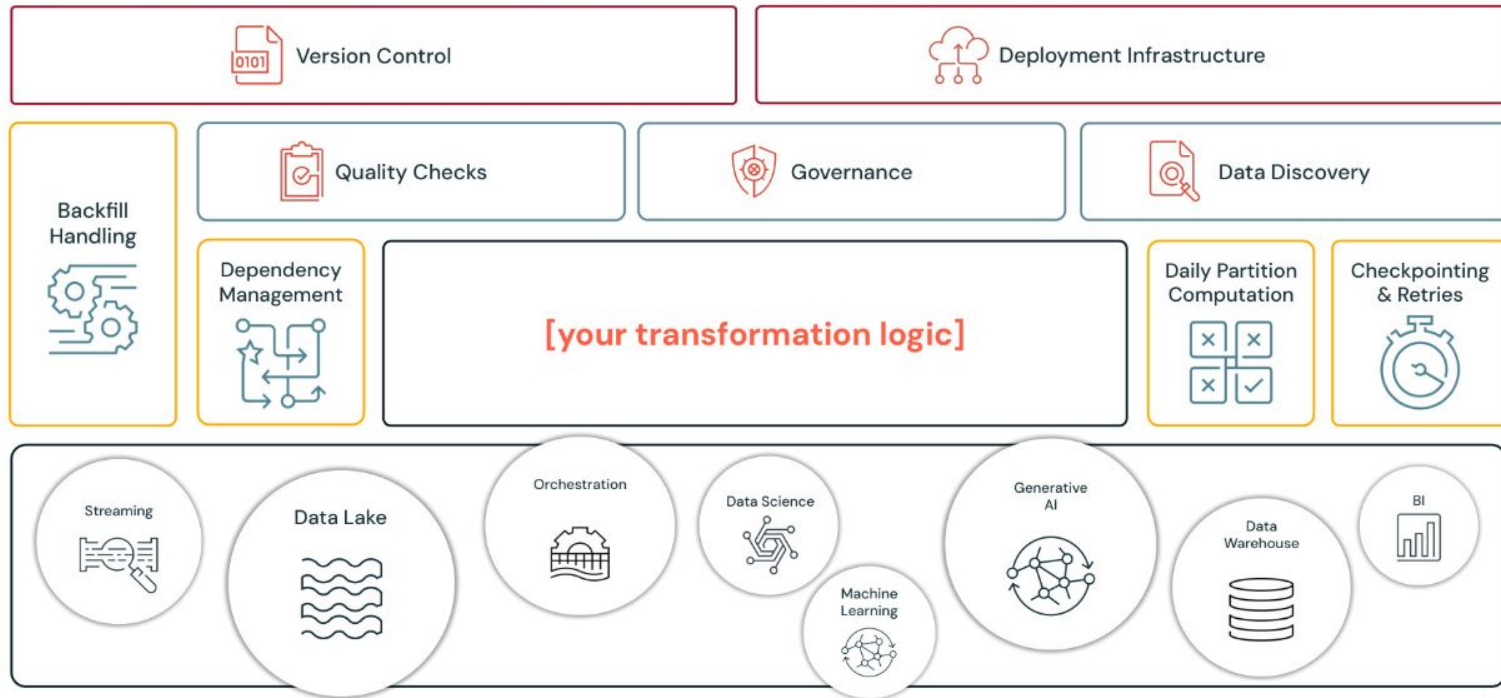
Simplifying batch and streaming ETL with automated reliability, optimizations and built-in data quality with **Lakeflow Spark Declarative Pipelines (SDP)**!



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

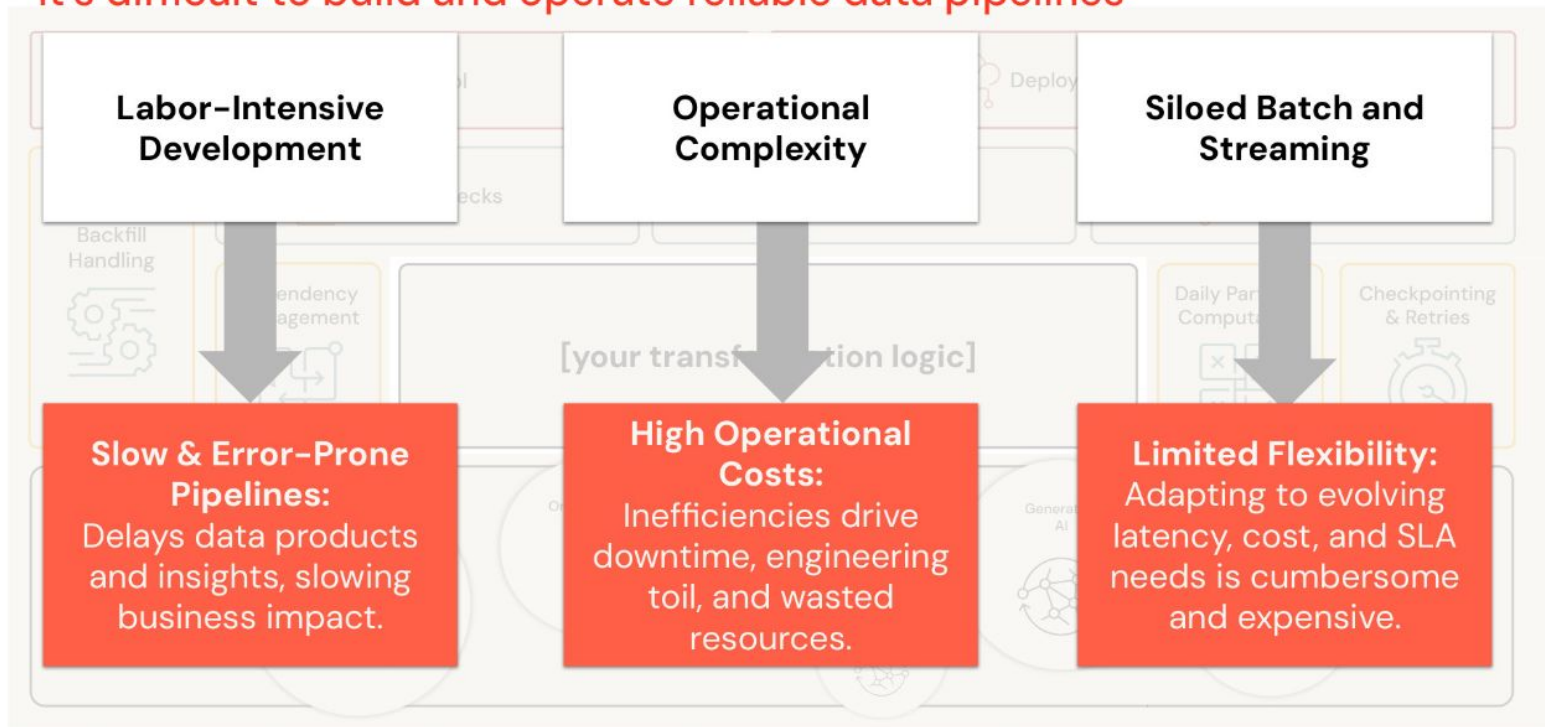
What are Spark Declarative Pipelines?

It's difficult to build and operate reliable data pipelines



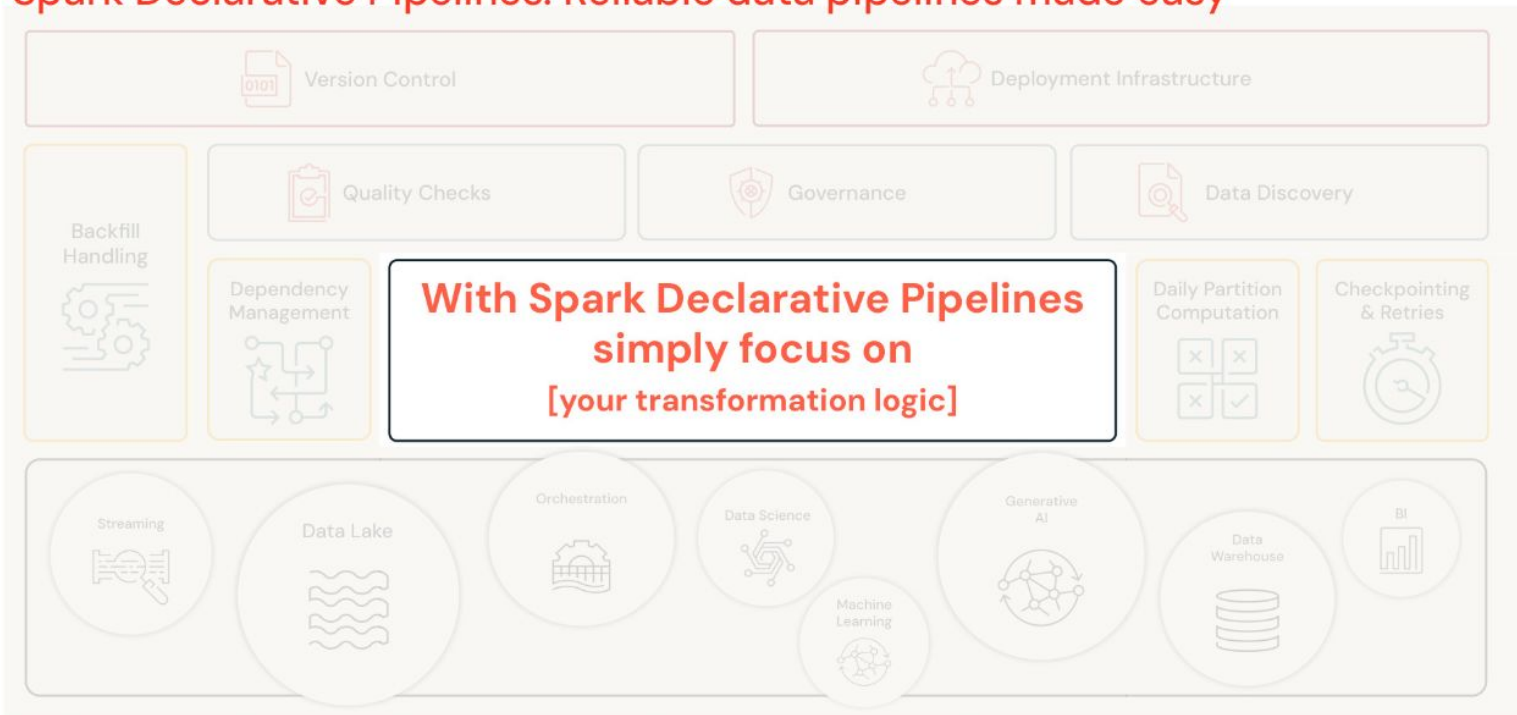
What are Spark Declarative Pipelines?

It's difficult to build and operate reliable data pipelines



What are Spark Declarative Pipelines?

Spark Declarative Pipelines: Reliable data pipelines made easy



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

What are Spark Declarative Pipelines?

Introducing Declarative Pipelines: Reliable data pipelines made easy

- **Simplified Pipeline Authoring**

Easily declare data ingestion and transformations with **SQL or Python**, and let Spark Declarative Pipelines handle the rest!

- **Intelligent Optimization at Scale**

Automated scaling and recovery improve reliability and reduce maintenance.

- **Unified Batch and Streaming**

Pipelines seamlessly adapt to **both near real-time and batch workloads**, optimizing performance and cost.

The screenshot displays the Databricks workspace interface for a declarative pipeline. On the left, a sidebar shows the 'Demo Pipeline' configuration, including a list of assets like 'sample_exploration', 'sample_transformations', and 'sample_trips_demo_pipeline.sql'. The main panel shows the 'sample_trips_demo_pipeline.sql' file with a SQL query defining a materialized view. The query is as follows:

```
1 -- This file defines a sample transformation.
2 -- Edit the sample below or add new transformations
3 -- using "+ Add" in the file browser.
4
5 CREATE MATERIALIZED VIEW sample_trips_demo_pipeline AS
6 SELECT
7   pickup_zip,
8   fare_amount
9 FROM samples.nyctaxi.trips
10
```

To the right of the code editor is a 'Pipeline graph' showing two nodes: 'sample_trips_demo...' and 'sample_zones_demo...'. Below the graph, a 'Tables' section provides a summary of the pipeline's output tables.

Name	Catalog	Schema	Type	Dura...	Writ...	Expectations	Drop...	Fail %	Flows
sample_trip...	labuser1043...	default	Materialized...	7s	-	Not defined	0	-	1
sample_zon...	labuser1043...	default	Materialized...	4s	-	Not defined	0	-	1

At the bottom, a status bar indicates the pipeline was last run on 5/29/2025 at 5:20:33 PM, with a duration of 1m 2s and a 'Refresh all' button.



Benefits of SDP

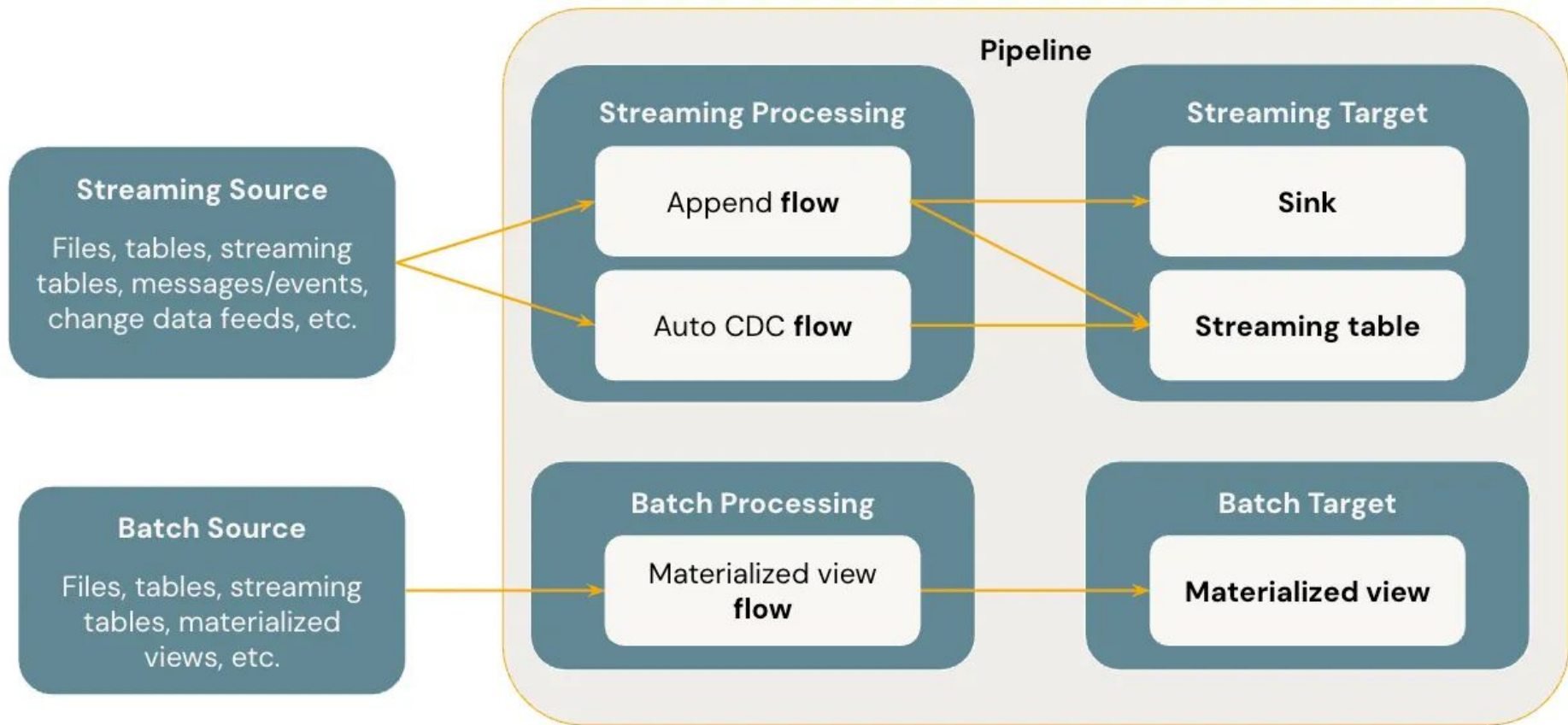
1. **Automatic orchestration:** SDP orchestrates processing steps (called "flows") automatically to ensure the correct order of execution and the maximum level of parallelism for optimal performance.
 - Additionally, pipelines automatically and efficiently retry transient failures.
 - The retry process begins with the most granular and cost-effective unit: the Spark task.
 - If the task-level retry fails, SDP proceeds to retry the flow, and then finally the entire pipeline if necessary.

Benefits of SDP

2. **Declarative processing:** SDP provides declarative functions that can reduce hundreds or even thousands lines of manual Spark and Structured Streaming code to only a few lines.
 - The SDP AUTO CDC API simplifies processing of Change Data Capture (CDC) events
 - It eliminates the need for manual code to handle out-of-order events, and it does not require an understanding of streaming semantics or concepts.

Benefits of SDP

3. **Incremental processing:** SDP provides an incremental processing (refresh) engine for materialized views.
 - To use it, you write your transformation logic with batch semantics, and the engine will only process new data and changes in the data sources whenever possible.
 - Incremental processing reduces inefficient reprocessing when new data or changes occur in the sources and eliminates the need for manual code to handle incremental processing.



What are Spark Declarative Pipelines?

Connecting to Data Sources

Lakeflow Connect



Cloud Storage
(S3, ADLS, GCS)



Message Queues
(Kafka, Pub/Sub, Kinesis, etc)



Databases
(SQL Server, PostgreSQL, etc)



Software as a Service
(Workday, Salesforce, etc)

Spark Declarative Pipelines



Bronze



Silver



Gold

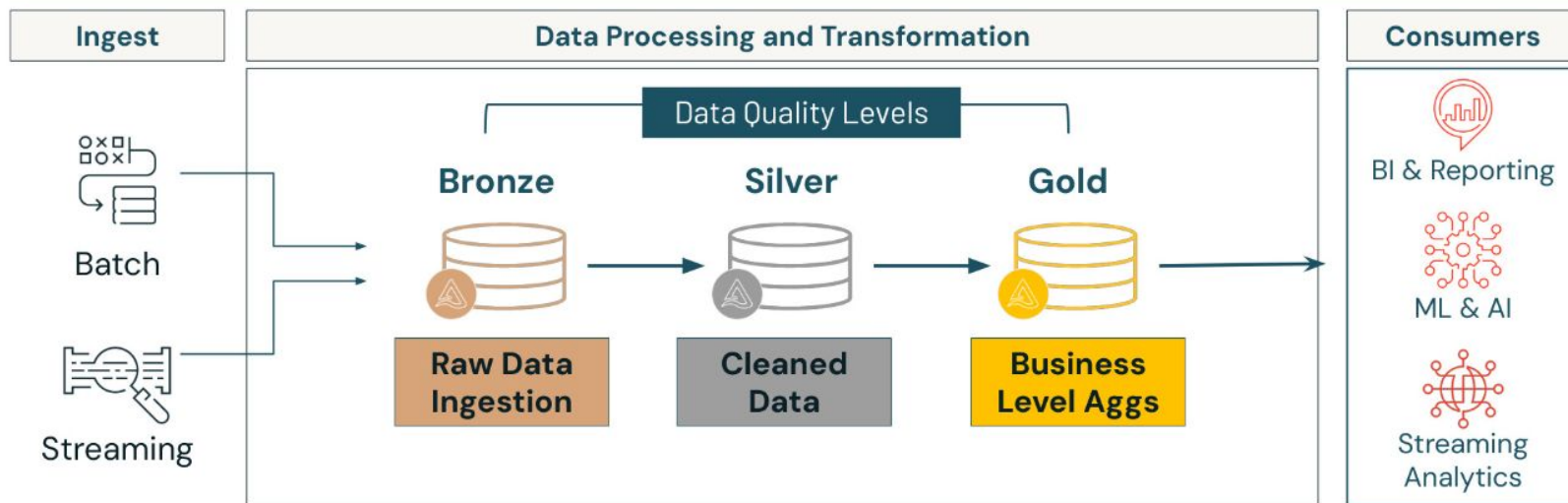


© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

What are Spark Declarative Pipelines?

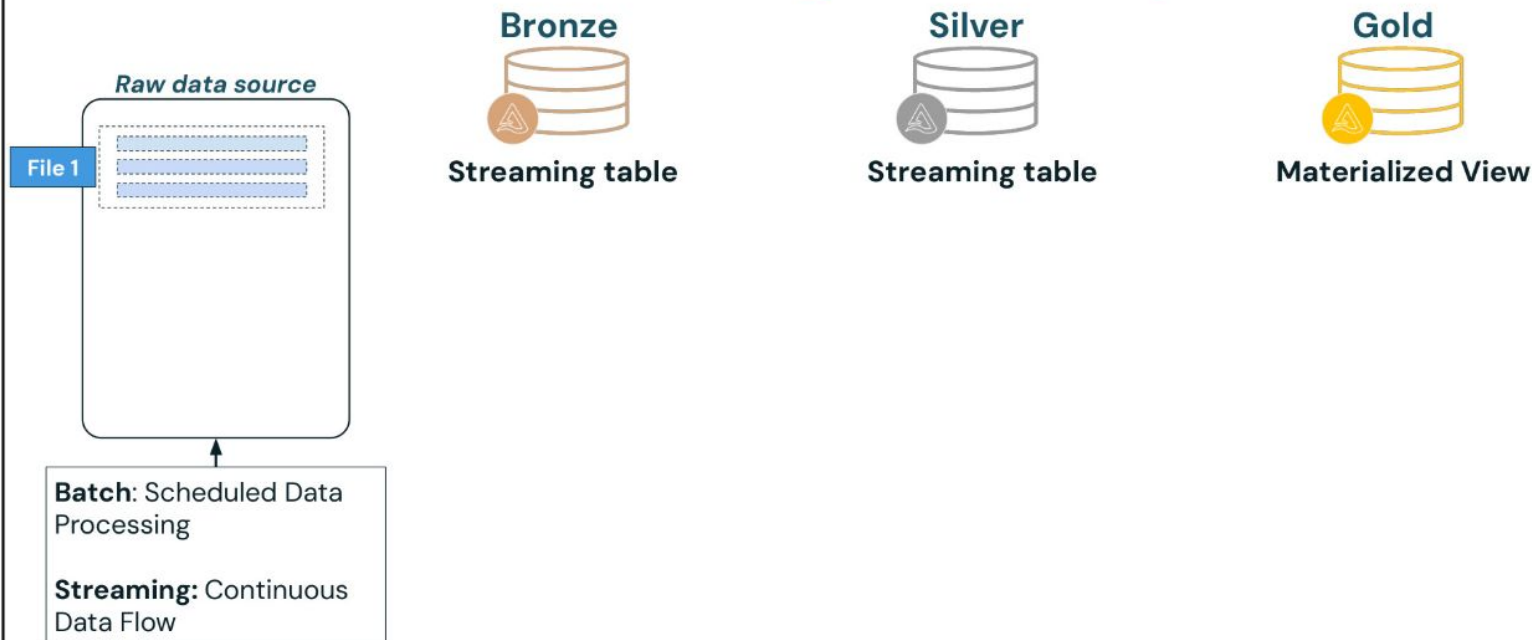
Simplifying Batch and Streaming ETL in the Medallion Architecture

Spark Declarative Pipelines



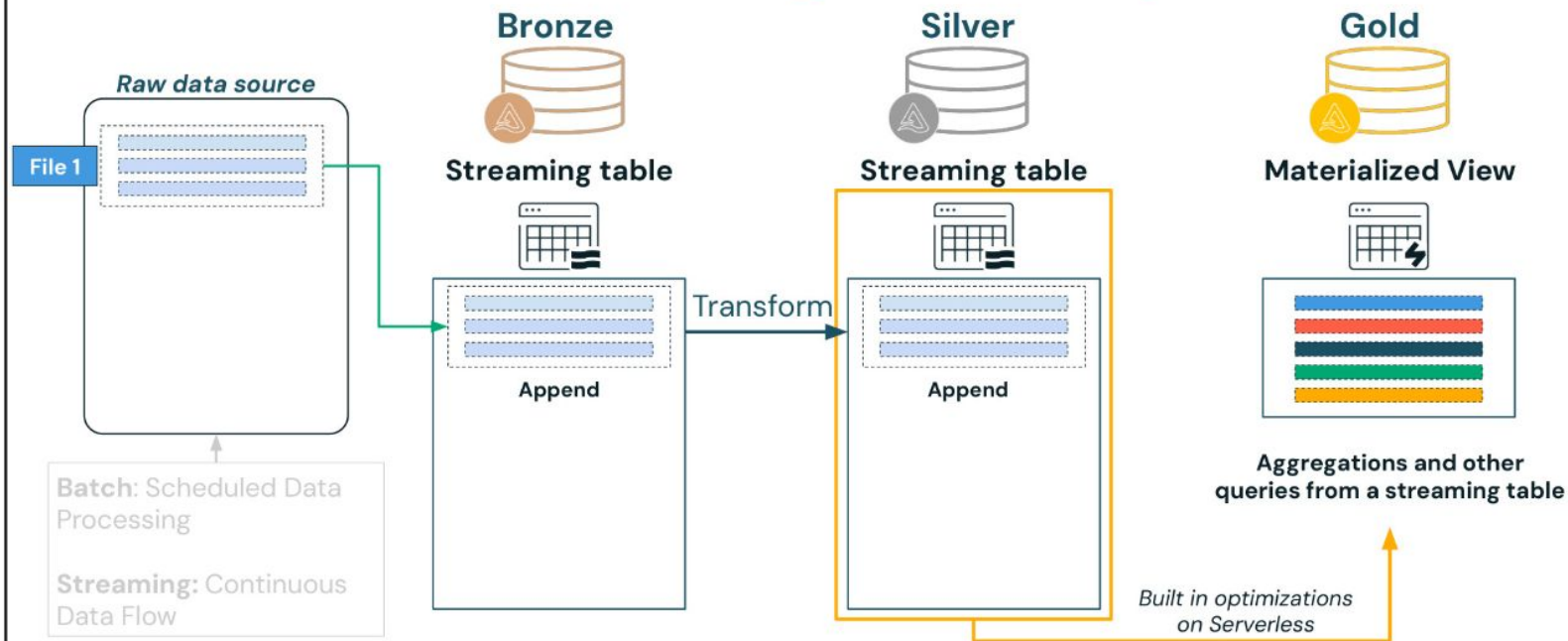
What are Spark Declarative Pipelines?

Overview of Incremental Processing in Declarative Pipelines



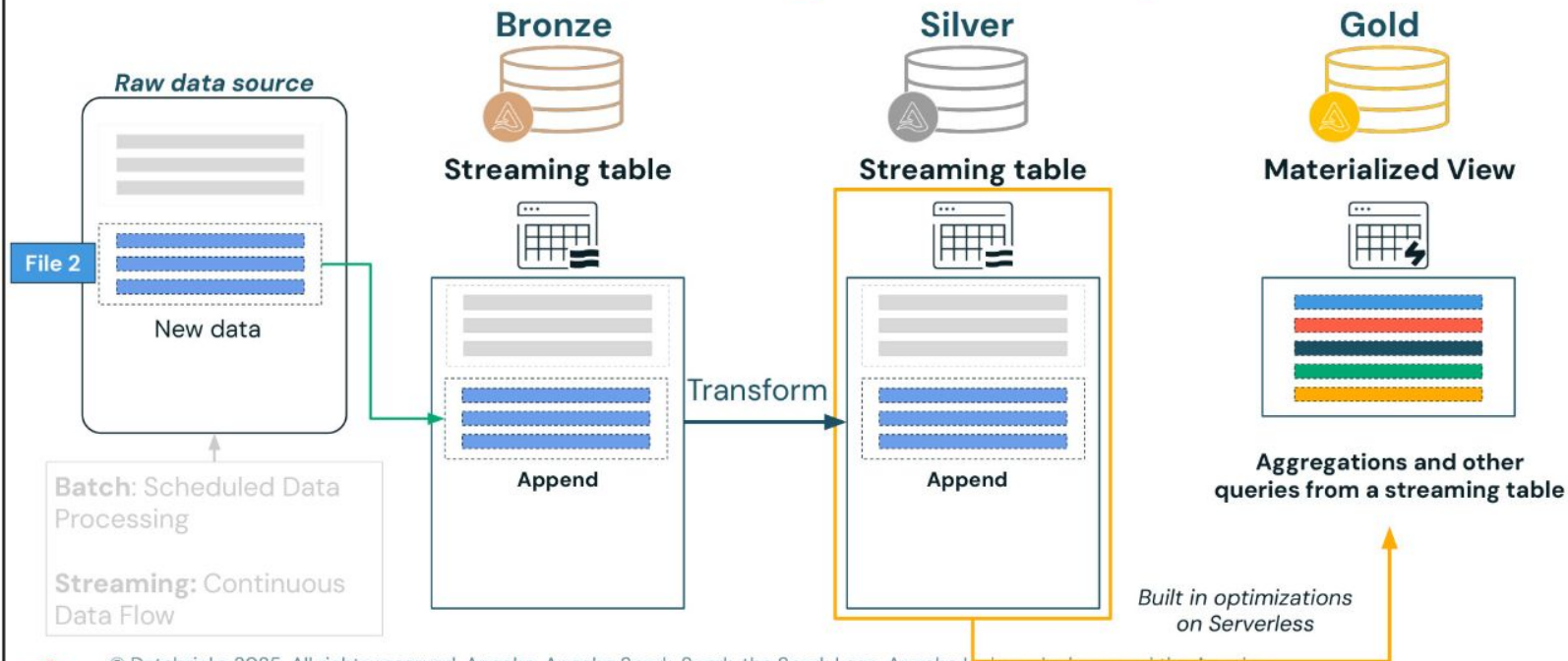
What are Spark Declarative Pipelines?

Overview of Incremental Processing in Declarative Pipelines (Run 1)



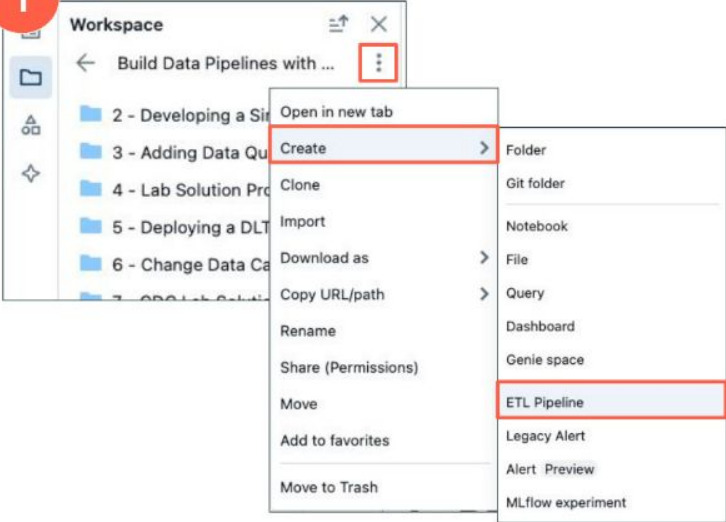
What are Spark Declarative Pipelines?

Overview of Incremental Processing in Declarative Pipelines (Run 2)



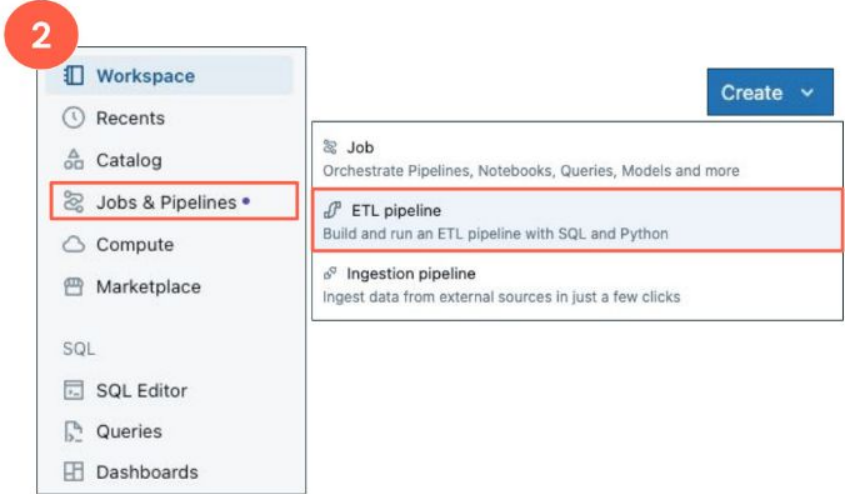
Creating a Spark Declarative Pipeline

1



The screenshot shows the Databricks Workspace interface. A red box highlights the three-dot menu icon in the top right corner of the workspace view. A red circle with the number '1' is next to it. A context menu is open, showing options like 'Open in new tab', 'Create', 'Clone', 'Import', 'Download as', 'Copy URL/path', 'Rename', 'Share (Permissions)', 'Move', 'Add to favorites', and 'Move to Trash'. The 'Create' option is highlighted with a red box. A secondary menu is open from 'Create', showing options like 'Folder', 'Git folder', 'Notebook', 'File', 'Query', 'Dashboard', 'Genie space', 'ETL Pipeline', 'Legacy Alert', 'Alert Preview', and 'MLflow experiment'. The 'ETL Pipeline' option is highlighted with a red box.

2



The screenshot shows the Databricks Workspace interface. A red circle with the number '2' is next to the 'Workspace' header. The 'Jobs & Pipelines' section is highlighted with a red box. A 'Create' button is visible in the top right corner. Below the 'Jobs & Pipelines' section, there are three options: 'Job', 'ETL pipeline', and 'Ingestion pipeline'. The 'ETL pipeline' option is highlighted with a red box. The 'ETL pipeline' option includes the text 'Build and run an ETL pipeline with SQL and Python'.

